

NATIONAL ENGINEERING CENTER
University of the Philippines

3.0

Agentic Workflows

Eugene Rex L. Jalao PhD

Professor

Department of Industrial Engineering and Operations Research

University of the Philippines Diliman

eljalao@up.edu.ph

UP NEC

Module Outline

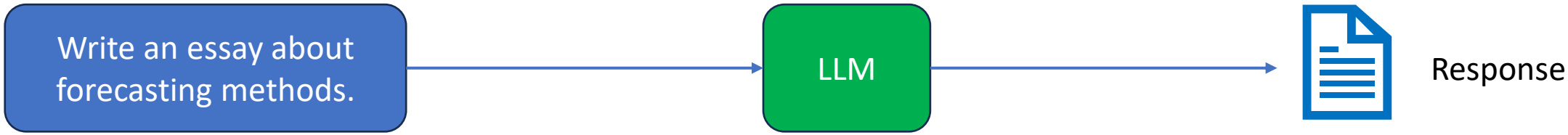
- Introduction to Agentic AI
  - Capstone Project Case Proposal Generation
- Deep Learning, Transformers and LLMs
  - Capstone Project Business Case Workflow
- **Agentic Workflows**
  - **Case Study and Capstone Project Work**
- Agentic RAGs
  - Case Study and Capstone Project Work
- Orchestration
  - Case Study and Capstone Project Work
- Stateful Multi-Agent Systems
  - Case Study and Capstone Project Work
- Evaluation
  - Case Study and Capstone Project Work
- LLMOps
  - Capstone Project Presentation

Outline for this Session

- Agentic AI
- Local LLM Architecture
- Prompt Design
- Structured Outputs
- Failure Modes and Guardrails
- Case Study

LLMs

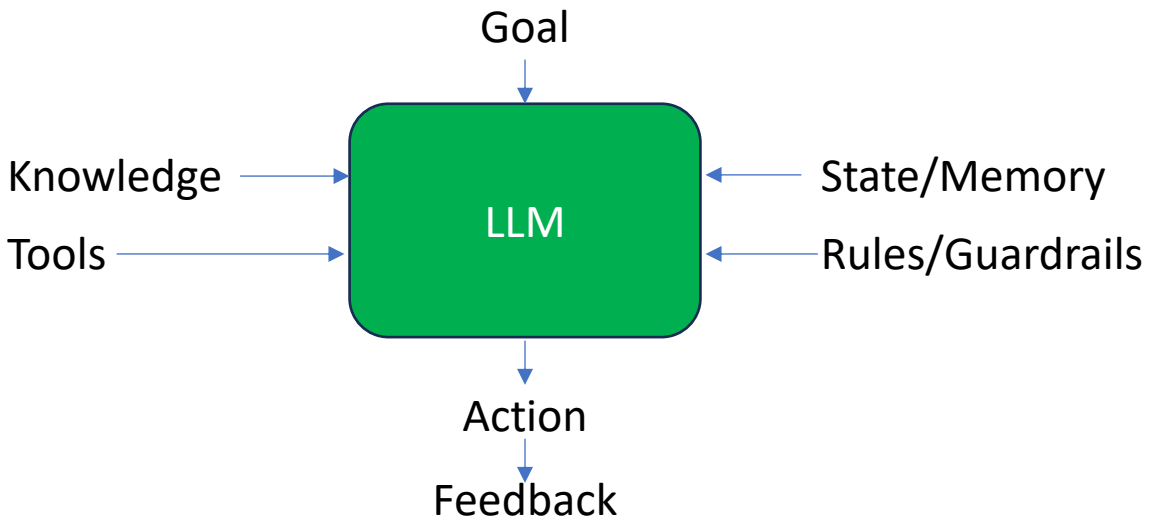
- **Definition:** A Large Language Model (LLM) is a type of artificial intelligence algorithm built on deep learning and the transformer architecture that understands, summarizes, generates, and predicts new text
- LLMs can answer, summarize, classify, and generate content.
- By themselves, they do not maintain workflow state or interact with external systems.
- Agentic systems add goals, tools, actions, state, and feedback.



https://www.techtarget.com/whatis/definition/large-language-model-LLM

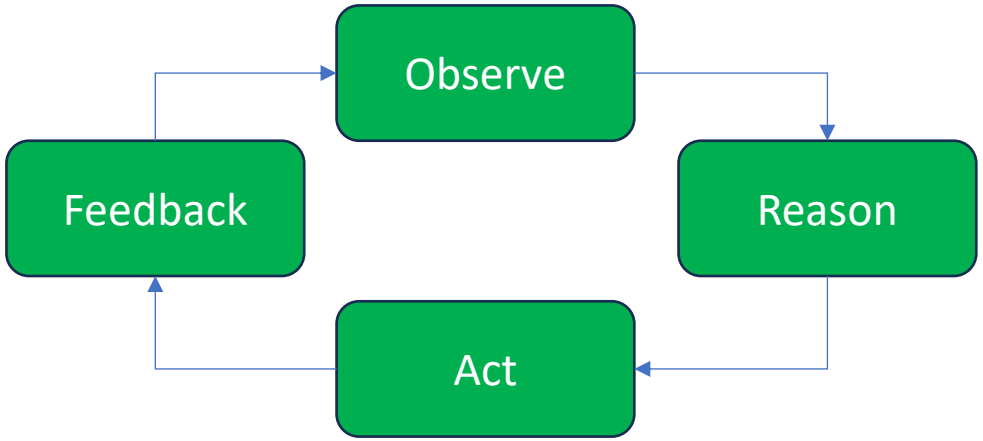
Building the Agentic Layer

- **Goals** define what the system is trying to accomplish.
- **Knowledge** provides relevant information needed for reasoning.
- **Tools** allow the system to interact with external applications and data.
- **State** records what has happened during the workflow.
- **Rules and guardrails** constrain what actions may be taken.
- **Feedback** allows the system to evaluate the result of an action and continue.



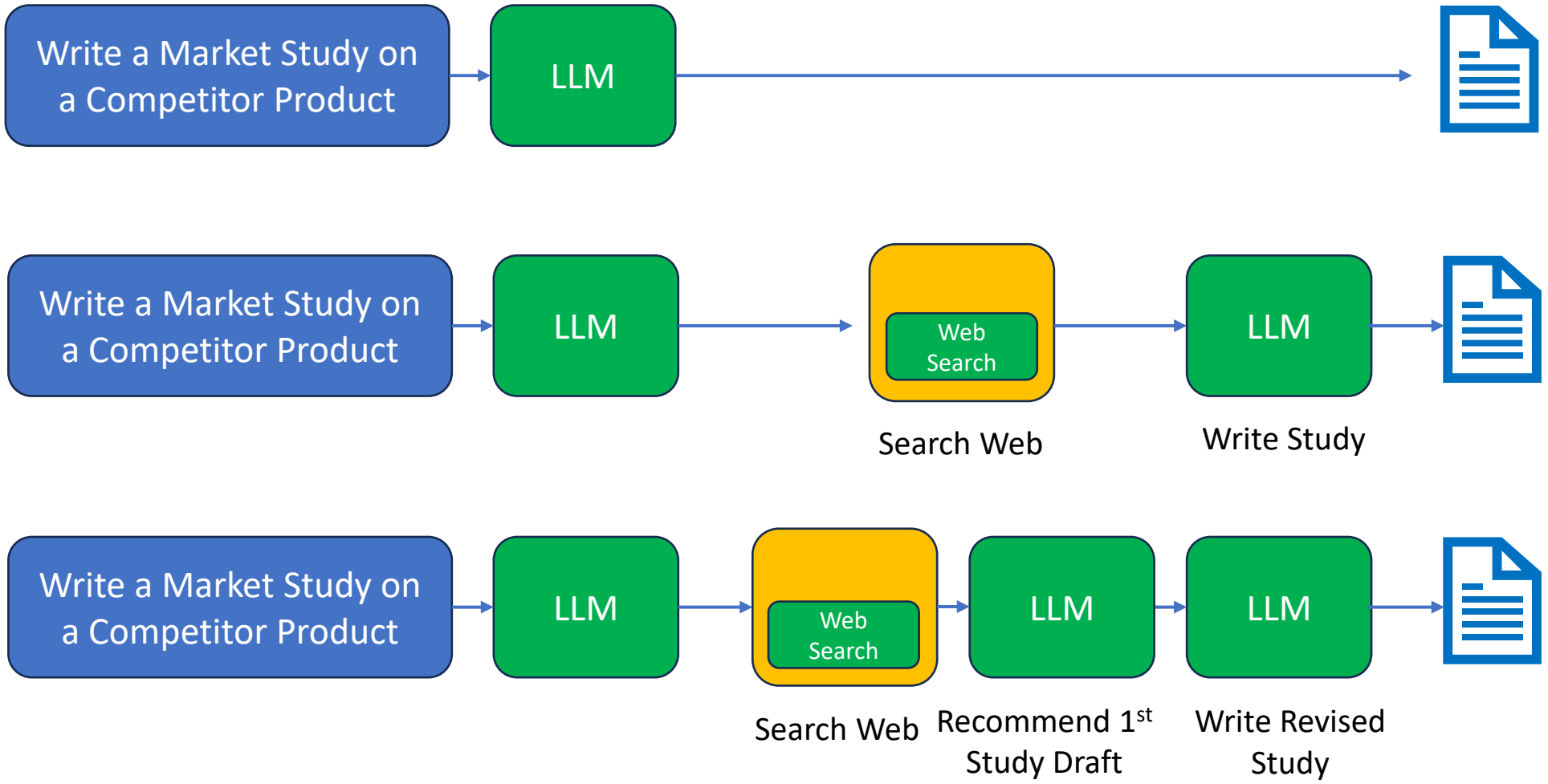
Agentic AI Operates as a Feedback Loop

- The agent **observes** the current task, context, and workflow state.
- An LLM **reasons** and determines an appropriate next action.
- The system executes or acts on the selected action using a tool or service.
- The resulting observation is added to the workflow state via feedback.
- The cycle continues until the goal is reached or intervention is required.



Task Decomposition

Example: Writing a Market Study

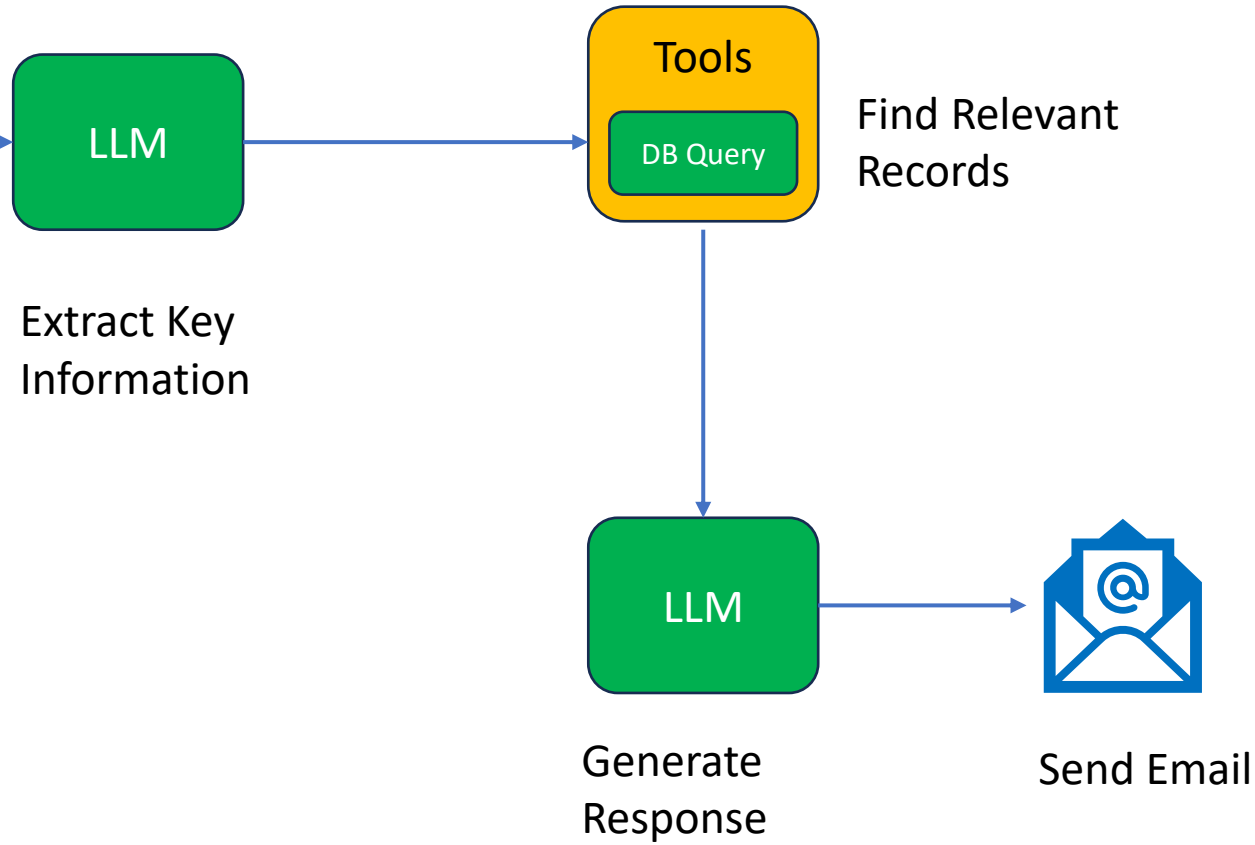


Email Response Agent

From: eljalao@upd.edu.ph
Subject: Follow Up Reimbursement DV

Hi, I would like to follow up on my request for Reimbursement via DV#12345. It has been 5 days stuck at the status of ”For Audit Validation.” Can I get an update for this?

Rex



Inventory of Building Blocks

Building Block	Examples	Use Case
Models	LLMS	Text Generation, Evaluator, Reasoning Agent
	Predictive Models	Forecasting, Classification
	Other AI Models	PDF-to-Text, Text-to-Speech
Tools	API	Web Search, Streaming Data, Email API
	Information Retrieval	RAG, Database Query
	Code Scripts	Calculator, Pandas Processing

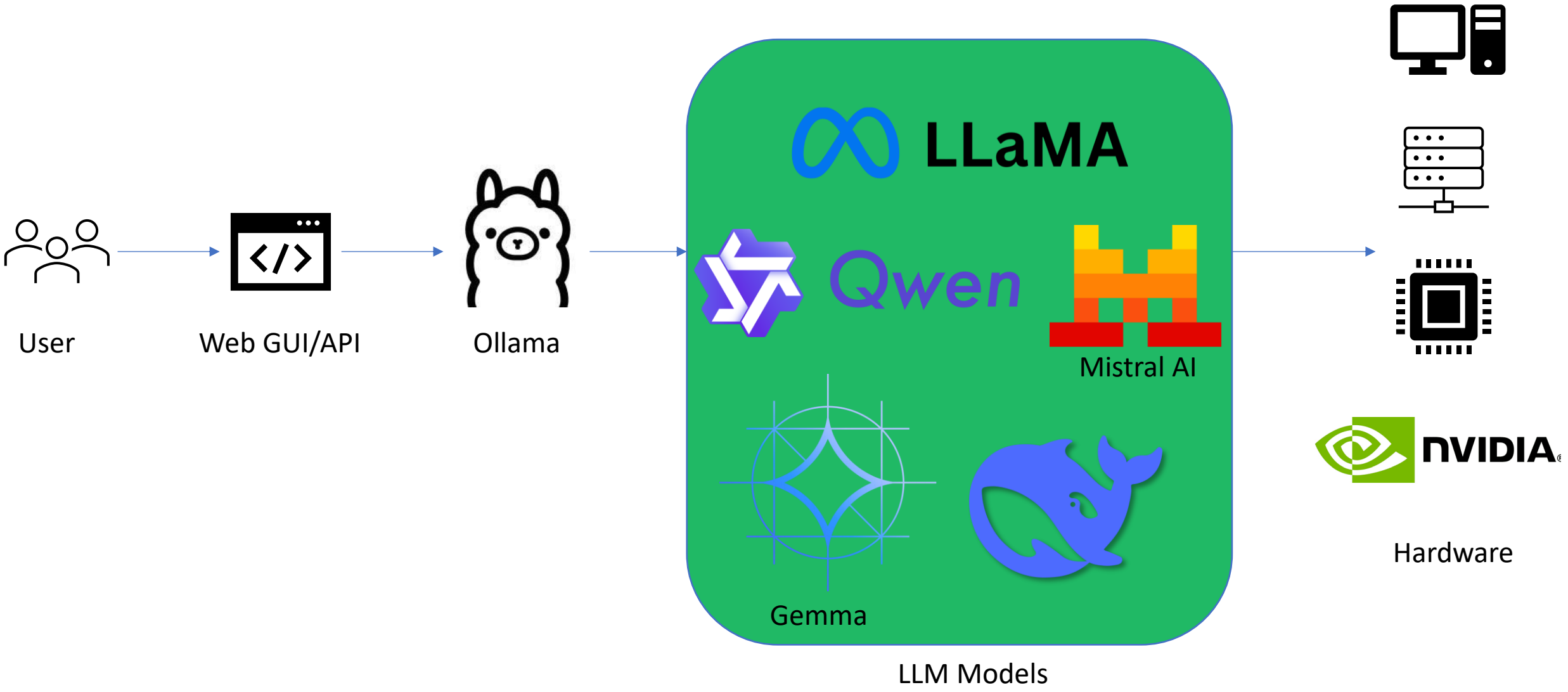
Outline for this Session

- Agentic AI
- **Local LLM Architecture**
- Prompt Design
- Structured Outputs
- Failure Modes and Guardrails
- Case Study

What Is a Local LLM?

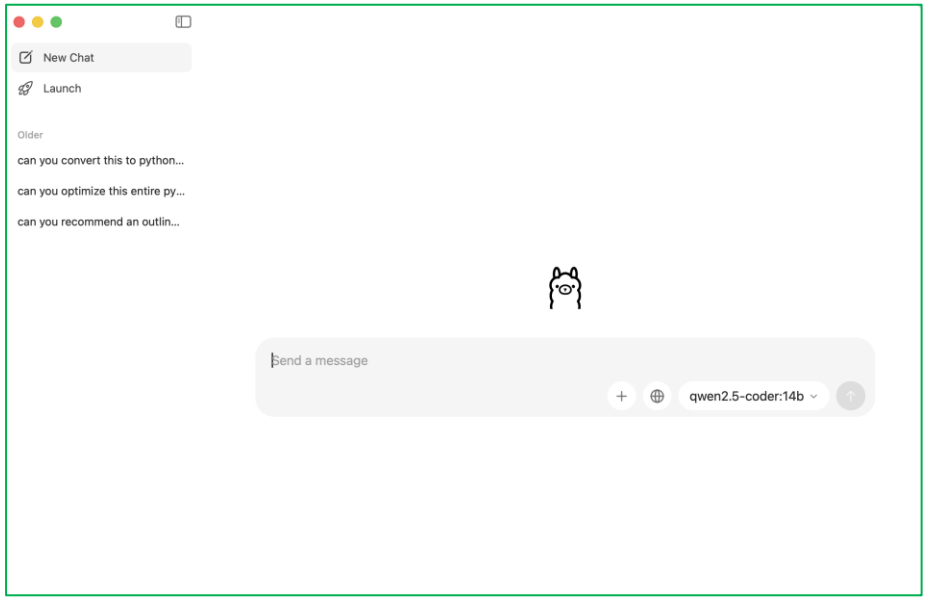
- **Definition:** A local LLM is an LLM which was downloaded and hosted on a local workstation, server, or private infrastructure.
- The model performs inference using the machine's CPU, GPU, and RAM.
- The application communicates with the local model through a runtime application such as Ollama.
- Local deployment can **reduce dependence** on external APIs and can provide greater control over data handling and model configuration.
- The organization remains responsible for hardware capacity, model management, security, and updates.

Basic Local LLM Architecture



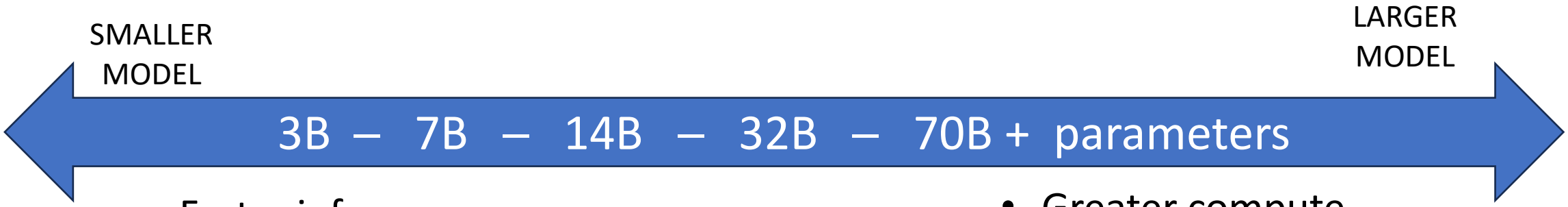
Ollama

- **Definition:** Ollama is an open-source platform that lets you download, manage, and run LLMs directly on your own computer (macOS, Windows, or Linux)
- It loads the selected LLM into available memory.
- It provides a simple API that applications can call.
- The application does not need to interact directly with the GPU to process model weights.



https://ollama.com/

Model Size, Memory, and Hardware



- Faster inference
- Lower memory requirement
- Laptop-friendly

- Greater compute demand
- Higher memory requirement
- Workstation/server oriented

Model Size, Memory, and Hardware

https://ollama.com/library/

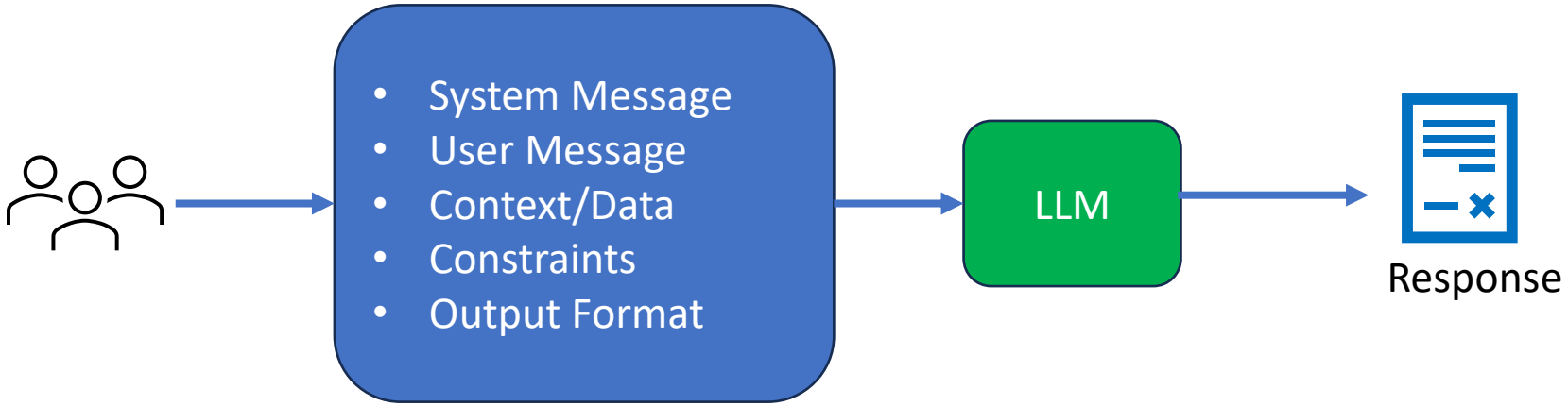
Model	Parameters	Approx. Model Size	Practical VRAM	Training Use
Qwen3 4B	4B	~2.5–2.6 GB	6–8 GB	Entry-level participant model
Gemma 3 4B	4B	~3.3 GB	6–8 GB	Small multimodal model
Qwen3 8B	8B	~5.2 GB	8–12 GB	Recommended participant model
Gemma 3 12B	12B	~8.1 GB	12–16 GB	Stronger multimodal exercises
Qwen3 14B	14B	~9.3 GB	16 GB	Advanced participant / instructor
Mistral Small 24B	24B	~14 GB	20–24 GB	Workstation demonstrations
Gemma 3 27B	27B	~17 GB	24 GB+	Advanced multimodal work
Qwen3 30B	30B	~19 GB	24 GB+	Advanced agentic reasoning
Qwen3 32B	32B	~20 GB	24–32 GB	High-end workstation
Llama 3.1 70B	70B	~43 GB	48–64 GB+	Server/high-end demonstration
Mixtral 8x22B	141B total / ~39B active	~80 GB	~96 GB+	Large MoE / function-calling demo
Qwen3 235B-A22B	235B total / ~22B active	~142 GB	~160 GB+	Large-scale reasoning and agent demo
Llama 3.1 405B	405B	~243 GB	~256–320 GB+	Multi-GPU / datacenter demonstration
Qwen3-Coder 480B	480B total / ~35B active	~290 GB	~300 GB+	Large agentic coding
DeepSeek-R1 671B	671B	~404 GB	~450–512 GB+	Large-scale reasoning / server-cluster demo

Outline for this Session

- Agentic AI
- Local LLM Architecture
- **Prompt Design**
- Structured Outputs
- Failure Modes and Guardrails
- Case Study

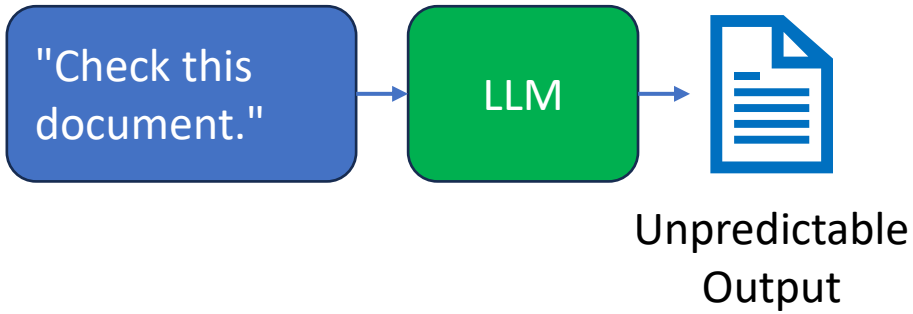
What Happens During an LLM Interaction?

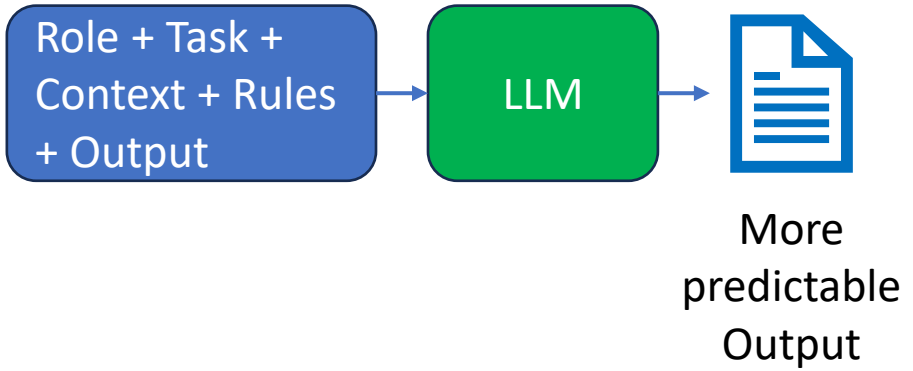
- The application provides instructions and context to the model.
- The model generates a response one token at a time
- Response quality depends on how the interaction is structured.
- Agentic applications require more predictable model behavior than ordinary chat.



Prompting

- **Definition:** A prompt is a natural language instruction, question, or block of context given to an AI model to guide its output.
- They define the task, context, constraints, and expected output.
- Good prompts reduce ambiguity and improve consistency.
- Agentic prompts should behave like clear software interfaces.
- Prompt designs should be tested using representative examples.





https://www.ionos.com/digitalguide/websites/web-development/llm-prompts/

The System Message

- **Definition:** The system prompt is part of the message sent to an LLM that establishes the model's role as a service-request classifier.
- It defines accepted categories, output format, and ambiguity handling.
- The same instructions can be reused across many requests.
- The system prompt is typically controlled by the application developer, not the end user.

The System Message

- **Example:** Sample System Prompt:
  - `You are an organizational service-request classifier.`
  - `Classify every request into exactly one of:`
  - `IT, Finance, HR, Procurement.`
  - `Return only the category.`
  - `Do not provide additional explanations.`
  - `If the request cannot be classified reliably,`
  - `return "Needs Review".`

Role

- **Definition:** A role tells the LLM what function it should perform in the application.
- A role defines the model's function and scope.
- Use task-specific roles rather than vague personas.
- Roles may differ across system components.
- Roles should complement, not replace, explicit instructions and rules

```
ROLE:
You are a procurement review
assistant.

Your responsibility is to:
- Check whether required documents
are complete.
- Identify missing requirements.
- Apply the supplied procurement
rules.
- Escalate uncertain cases for
human review.
```

The User Message

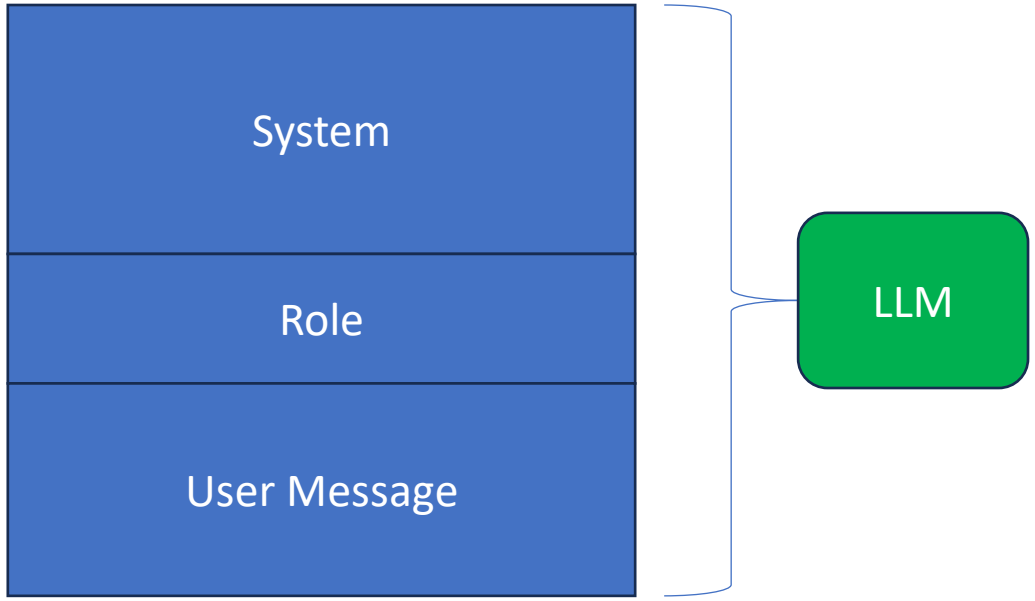
- **Definition:** The message is part of the prompt that provides a specific task or request that the LLM must perform for the current interaction.
- The user prompt provides the specific information to analyze.
- It reuses rules already defined in the system prompt.
- This separation reduces duplication and makes prompts easier to maintain.

User Prompt

- **User Request 1**
  - My laptop cannot connect to the office Wi-Fi.
  - Expected:
  - IT
- **User Request 2**
  - My reimbursement from last month has not been paid.
  - Expected:
  - Finance
- **User Request 3**
  - I need a certificate of employment.
  - Expected:
  - HR

Why Separate System and User Prompts?

- A separate system prompt keeps rules consistent across many requests.
- It reduces repetition and makes updates easier.
- It separates trusted application instructions from user input.
- It also supports easier testing and later use of tools, routing, structured outputs, and guardrails.



Context: What Does the Model Know?

- An LLM works from its learned parameters and **current context**.
- Context can include prompts, examples, documents, tool results, workflow state, and RAG content.
- Context size is limited, and irrelevant information can reduce response quality.
- Sources:
  - System Instructions
  - Previous Conversation
  - Current User Request
  - Retrieved Documents
  - Tool Results
  - Workflow State

gemma3:4b

- **Context Window (Input/Processing):** Up to 128,000 tokens ~ Paperback / Book pages: ~300 pages.
- **Maximum Output Tokens:** 8,192 tokens ~16-20 pages.

Context: What Does the Model Know?

- **Example:**
- Purchase Request Number: PR-2026-0145
- Requesting Unit: College of Engineering
- Supplier: ABC Technologies
- Amount: ₱850,000
- Transaction Date: September 15, 2026
- Required Documents: - Purchase Request - Supplier Quotation – Approval Form
- Available Documents: - Purchase Request –
- Approval Form Purpose: Purchase of 10 laptop computers for the engineering laboratory.

Separate Instructions from Context

- **Instructions** tell the model what to do.
- **Context** contains the information on which the task should operate.
- Mixing instructions and source data can make prompts harder to interpret.
- Clear delimiters help separate documents from instructions.
- Retrieved RAG content should normally be treated as context rather than instructions.

Constraints

- **Definition:** Constraint is an explicit rule, boundary, or limitation built into a prompt to control the content, length, format, style, or safety of a LLM's output.
- Constraints can encode business rules and safety requirements.
- Explicit instructions can reduce unsupported assumptions.
- The model should be told what to do when information is missing.
- Important deterministic rules may still be better enforced by application code rather than relying only on the LLM.

https://codesignal.com/learn/courses/prompting-foundations/lessons/defining-constraints-and-requirements-for-effective-prompts

Constraints

- **Example:**
- Review this transaction.

- RULES:
- 1. Use only the supplied information.
- 2. Do not assume that missing documents exist.
- 3. If required information is missing, return REVIEW.
- 4. Transactions above ₱1,000,000 require human approval.
- 5. Do not approve a transaction if the amount is unknown.

What to Do When It Does Not Know

- LLMs may attempt to provide an answer even when information is **incomplete**.
- Prompts should define what the model should do when evidence is missing.
- The model should be allowed to return UNKNOWN or INSUFFICIENT_INFORMATION.
- Unsupported assumptions should not be treated as facts.
- Low-confidence or incomplete cases can be routed to a human reviewer.
- Critical business decisions should not depend solely on the model's confidence score.

Local LLMs

- **Exercise 1: Unstructured vs Structured Prompting with a Local LLM**
- **Objectives**
  - Send a purchase request from Python to an Ollama-hosted local LLM.
  - Use the components **System Message, User Message, Context/Data, Constraints, and Output Format**.
  - Observe how prompt structure improves consistency and predictability.
  - Restrict the structured response to approved workflow values.
  - Observe the flow from **application** → **Ollama** → **model** → **application**.
- **Business Case**
  - A university receives a purchase request for **20 laptop computers costing PHP 1,200,000**. Purchase requests above **PHP 1,000,000** require human review; all others follow standard review.
- The Purchase Request Review Assistant must return only:
  - HUMAN_REVIEW or STANDARD_REVIEW

Local LLMs

```python
#%% EXERCISE 1: UNSTRUCTURED VS STRUCTURED PROMPTING

import ollama

model_name = "gemma3:4b"

# ------------------------------------------------------------
# 1. PURCHASE REQUEST DATA
# ------------------------------------------------------------

request_id = "PR-2026-001"
amount = 1200000

purchase_request = f"""
Request ID: {request_id}
Item: 20 laptop computers
Amount: PHP {amount:,}
Purpose: Replace obsolete laboratory computers.
"""

# ------------------------------------------------------------
# 2. SHOW LOCAL LLM ARCHITECTURE
# ------------------------------------------------------------

print("========================================")
print("LOCAL LLM ARCHITECTURE")
print("========================================")

print("Python Application")
print("       ↓")
print("  ollama.chat()")
print("       ↓")
print(" Ollama Runtime")
print("       ↓")
print(" " + model_name)
```

```
========================================
LOCAL LLM ARCHITECTURE
========================================
Python Application
       ↓
  ollama.chat()
       ↓
 Ollama Runtime
       ↓
 gemma3:4b

========================================
PART A: UNSTRUCTURED PROMPT
========================================

PROMPT:
Review this purchase request and tell me what should happen:


Request ID: PR-2026-001
Item: 20 laptop computers
Amount: PHP 1,200,000
Purpose: Replace obsolete laboratory computers.

MODEL RESPONSE:
Okay, here's a breakdown of what should happen regarding Purchase Request
PR-2026-001, along with the typical steps involved:

**1. Initial Review & Approval (Immediate - Within 1-2 Days):**

* **Department Head Review:** The request *must* be reviewed and approved
department head responsible for the laboratory. This is the first gatekeep
ensure the need is legitimate and the amount is reasonable. They'll likely
```

```
========================================
PART B: STRUCTURED PROMPT
========================================

SYSTEM MESSAGE:
You are a Purchase Request Review Assistant.
Your task is to determine the correct review path
for a purchase request.

USER MESSAGE:
Review the following purchase request.


PURCHASE REQUEST:

Request ID: PR-2026-001
Item: 20 laptop computers
Amount: PHP 1,200,000
Purpose: Replace obsolete laboratory computers.



BUSINESS RULE:

- If the purchase amount is greater than PHP 1,000,000,
  select HUMAN_REVIEW.
- Otherwise, select STANDARD_REVIEW.

Do not invent information.
Use only the purchase request provided.



Return exactly one of these values:

HUMAN_REVIEW
STANDARD_REVIEW

Do not include explanations.

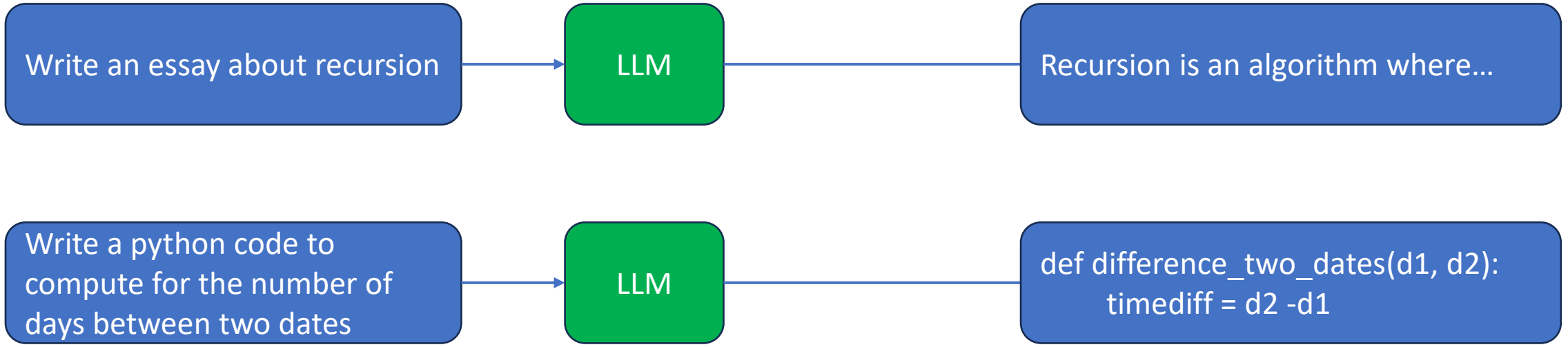
MODEL RESPONSE:
```

Prompting Techniques and Prompt Patterns

Layer	Question	Examples
Prompting Techniques	How should I instruct the model?	Zero-shot, few-shot, role prompting, constraints, examples
Prompt Patterns	What job should the model perform?	Classification, extraction, summarization, routing, tool selection

Zero Shot Prompts

- Direct Generation



Zero Shot

- **Example:**
- You are a procurement review assistant.
- TASK:
- Classify the request as APPROVE or REVIEW.
- RULES:
- - If any required document is missing, return REVIEW.
- - If the amount is above ₱1,000,000, return REVIEW.
- - Otherwise, return APPROVE.
- REQUEST:
- Amount: ₱750,000
- Documents: Complete
- OUTPUT:
- Return the decision and a short reason.



Decision:
APPROVE
Reason:
Documents are complete and the amount is within the threshold.

One Shot

- **Example:**
- You are a procurement review assistant.
- Classify each request as APPROVE or REVIEW.
- RULES:
- - Missing document → REVIEW
- - Amount > ₱1,000,000 → REVIEW
- - Otherwise → APPROVE
- EXAMPLE:
- Amount: ₱500,000
- Documents: Missing Supplier Quotation
- Decision: REVIEW
- Reason: A required document is missing.
- NOW REVIEW:
- Amount: ₱850,000
- Documents: Complete



Decision: APPROVE
Reason: The required documents are complete and the amount is within the threshold.

Few Shot

- **Example:**
- You are a procurement review assistant.
- Classify each request as APPROVE or REVIEW.
- EXAMPLE 1:
- ₱400,000 + Complete Documents
- → APPROVE
- EXAMPLE 2:
- ₱650,000 + Missing Supplier Quotation
- → REVIEW
- EXAMPLE 3:
- ₱1,500,000 + Complete Documents
- → REVIEW
- NOW REVIEW:
- ₱900,000 + Missing Approval Form
- Decision:



Decision: REVIEW
Reason: A required document is missing.

Three Prompting Strategies

- **Exercise 2: Classifying Purchase Requests**
- Objectives:
  - Construct zero-shot, one-shot, and few-shot prompts.
  - Observe how examples influence the model's interpretation of a task.
  - Compare consistency across the three prompting strategies.
  - Understand when examples are useful for ambiguous classification tasks.
  - Recognize that adding examples increases prompt length and context usage.
- Business Case:
  - An organization receives purchase requests for many different types of items and services. Before processing, each request must be assigned to one of four categories:ICT Equipment, Office Supplies, Laboratory Equipment, Professional Services

Three Prompting Strategies

- **Exercise 2: Classifying Purchase Requests**

```
#%% EXERCISE 2: ZERO-SHOT, ONE-SHOT, AND FEW-SHOT PROMPTING

import ollama

model_name = "gemma3:4b"

# ------------------------------------------------------------
# 1. PURCHASE REQUEST TO CLASSIFY
# ------------------------------------------------------------

purchase_request = """
Request ID: PR-2026-002
Item: 10 high-performance desktop workstations
Purpose: Upgrade the university AI and data science laboratory.
"""

categories = """
ICT Equipment
Office Supplies
Laboratory Equipment
Professional Services
"""

# ------------------------------------------------------------
```

```
========================================
PURCHASE REQUEST
========================================
Request ID: PR-2026-002
Item: 10 high-performance desktop workstations
Purpose: Upgrade the university AI and data science laboratory.

========================================
ZERO-SHOT RESULT
========================================
ICT Equipment

========================================
ONE-SHOT RESULT
========================================
ICT Equipment

========================================
FEW-SHOT RESULT
========================================
ICT Equipment
```

Prompting Patterns

- Classification
- Information Extraction
- Summarization
- Grounded Question Answering
- Routing
- Tool Selection

Prompting Pattern: Classification

- **Purpose**: Assign an input to one predefined category.
- **Good classification prompts specify**
  - Allowed categories
  - Clear category definitions when needed
  - Exactly one or multiple labels
  - Required output format
  - What to do when uncertain
- **Common uses**
  - Ticket classification
  - Email classification
  - Sentiment classification
  - Document classification
  - Risk categorization

Prompting Pattern: Classification

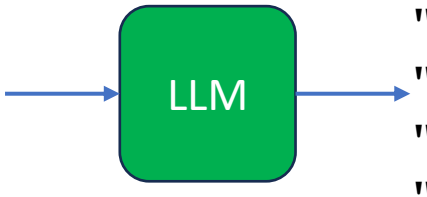
- **Example:** Prompt
  - `Classify the following document into exactly one category:`
    - `PURCHASE_REQUEST`
    - `PURCHASE_ORDER`
    - `INVOICE`
    - `DELIVERY_RECEIPT`
  - `Document:`
    - `"The supplier submitted a request for payment of ₱85,000`
    - `for computer equipment that has already been delivered."`
  - `Return only the category.`
- Expected Output:
  - `INVOICE`

Information Extraction

- **Purpose**: Convert unstructured text into structured data.
- Information extraction prompting asks the LLM to identify specific information in unstructured text and convert it into structured fields.
- **Common uses**
  - Forms
  - Emails
  - Reports
  - Contracts
  - Service requests
  - Transaction documents

Information Extraction

- **Example:** Prompt
  - Extract the supplier, document date, item, quantity, and amount from the following text.
  - Return the result as JSON.
  - Use null if a value is not provided.
  - Text:
    - "ABC Technologies submitted an invoice dated September 10, 2026 for 10 laptop computers worth ₱850,000."



- Expected Output:

```
{
  "supplier": "ABC Technologies",
  "date": "2026-09-10",
  "item": "Laptop computers",
  "quantity": 10,
  "amount": 850000
}
```

Prompting Pattern: Summarization

- **Purpose:** Summarization prompting asks the LLM to reduce a larger amount of information while preserving the important meaning.
- **Summarization can be controlled by specifying**
  - Length
  - Audience
  - Required sections
  - Information to include
  - Information to exclude
  - Output format

Prompting Pattern: Summarization

- **Example:** Prompt
  - Summarize the following purchase justification in no more than three sentences.
  - Preserve:
  - - the requested item,
  - - the reason for the purchase,
  - - the amount, and
  - - the expected benefit.
  - Purchase justification:
  - "The department is requesting the purchase of 10 new laptop computers costing ₱850,000. The existing computers are more than six years old and are unable to efficiently run current engineering, data-analysis, and simulation software.
  - The new laptops will support faculty research, administrative work, and student projects and are expected to improve staff productivity."



- Output:
  - The department is requesting 10 laptop computers costing ₱850,000 to replace outdated equipment. The existing computers cannot adequately support current engineering and data-analysis applications. The purchase is expected to improve productivity and support research and student projects.

Prompting Pattern: Grounded Question Answering

- **Purpose**: Grounded question answering asks the LLM to answer a question using only supplied or retrieved evidence.
- If evidence is insufficient, abstain rather than guess.

Prompting Pattern: Grounded Question Answering

- **Example:** Prompt
- Answer the question using only the policy provided below.
- If the policy does not contain enough information,
- respond with:
- INSUFFICIENT INFORMATION
- Policy:
- "Purchases above ₱1,000,000 require approval
- from the University President."
- Question:
- Does a purchase amounting to ₱850,000 require
- approval from the University President?
- Briefly explain your answer using the policy.



Expected:

No. The policy requires approval from the University President only for purchases above ₱1,000,000.

Prompting Pattern: Routing

- **Purpose**: Determine which workflow, specialist, or processing path should handle a request.
- Routing differs from classification. Classification asks what is this?, Routing asks where should this go next?
- They may use similar prompts, but routing affects the next application step.

Prompting Pattern: Routing

- **Example:** Prompt:
  - `Determine which workflow should process the`
  - `following user request.`
  - `Choose exactly one route:`
  - `DOCUMENT_REVIEW`
  - `PAYMENT_STATUS`
  - `PROCUREMENT`
  - `HR_REQUEST`
  - `GENERAL_INQUIRY`
  - `User request:`
  - `"Can you check whether my reimbursement has`
  - `already been processed?"`
  - `Return the selected route only.`



Expected:
`PAYMENT_STATUS`

Prompting Pattern: Tool Selection

- **Purpose**: Ask the LLM to propose which approved tool or function should be used.
- The model proposes the action; the application controls whether that action is allowed and executed.

Prompting Pattern: Tool Selection

- **Example:** Prompt
  - You may use only the following approved functions:

  - get_payment_status(dv_number)
  - search_purchase_order(po_number)
  - get_supplier_information(supplier_name)
  - send_notification(recipient, message)

  - User request:
  - "Has payment for DV-2026-1055 already been released?"

  - Select the most appropriate function.

  - Return JSON containing:
  - - tool
  - - arguments



Expected:

```
{
"tool":
"get_payment_sta
tus",
"arguments": {
"dv_number":
"DV-2026-1055" }
 }
```

Content Generation Prompting

- **Purpose:** Content generation prompting asks the LLM to transform information into clear text intended for a human reader.
- Examples:
  - **Drafting**
    - Create a professional response to the user based on the following validated record.
  - **Rewriting**
    - Rewrite this message in a professional tone.
  - **Translation**
    - Translate this message into Filipino. Preserve names and numbers exactly.
  - **Format conversion**
    - Convert the following notes into a formal memo.

Content Generation Prompting

- **Example:** Prompt
  - Write a short and professional notification
  - to the requester using the following information.

  - DV Number: DV-2026-1055
  - Status: Returned
  - Reason: Missing supplier quotation

  - Tell the requester:
  - - why the DV was returned, and
  - - what action is required before resubmission.

  - Use no more than three sentences.



Expected:

DV-2026-1055 has been returned because the required supplier quotation was not attached. Please attach the supplier quotation and resubmit the DV for review.

Choosing the Right Prompting Pattern

- Different prompts give the LLM different **roles** inside an Agentic AI system: classify, extract, summarize, reason from evidence, route, act, and communicate.
- **Lower temperature** is generally preferable when the model is being used for classification, extraction, routing, or structured decision support.

Strategy	What the LLM does	Typical output
Classification	Assigns a category.	INVOICE
Extraction	Converts text into fields.	JSON
Summarization	Reduces information.	Short summary
Grounded QA	Answers from supplied evidence.	Evidence-based answer
Routing	Chooses the next workflow.	PAYMENT_STATUS
Tool Selection	Chooses an approved function.	Function + arguments
Generation	Produces human-readable content.	Message, explanation, report

Prompt Designs

- **Exercise 3 — Applying Common Prompting Patterns**
- Participants will use a local LLM to apply several common prompting patterns:
  - **Classification** — assign an input to a predefined category.
  - **Information Extraction** — convert unstructured text into structured fields.
  - **Summarization** — reduce information while preserving important details.
  - **Grounded Question Answering** — answer using only supplied evidence.
  - **Routing** — determine which workflow should handle a request.
  - **Tool Selection** — propose an approved function and its arguments.
  - **Content Generation** — transform validated information into human-readable text.
- Participants will observe how the **prompt structure changes depending on the application goal**, even when the same LLM is used.

Prompt Designs

- **Exercise 3 — Applying Common Prompting Patterns**

```python
#%% EXERCISE 3: COMMON PROMPTING PATTERNS

import ollama

model_name = "gemma3:4b"


# ------------------------------------------------------------
# HELPER FUNCTION
# ------------------------------------------------------------

def ask_model(prompt):

    response = ollama.chat(
        model=model_name,
        messages=[
            {
                "role": "user",
                "content": prompt
            }
        ],
        think=False,
        options={
            "temperature": 0
        }
    )

    return response["message"]["content"].strip()


# ============================================================
# PATTERN 1: CLASSIFICATION
# ============================================================

classification_prompt = """
Classify the following document into exactly one category:

PURCHASE_REQUEST
PURCHASE_ORDER
INVOICE
DELIVERY_RECEIPT
```

```
============================================================
PATTERN 1: CLASSIFICATION
============================================================

PURPOSE:
Assign an input to one predefined category.

RESULT:
INVOICE

============================================================
PATTERN 2: INFORMATION EXTRACTION
============================================================

PURPOSE:
Convert unstructured text into structured data.

RESULT:
```json
{
  "supplier": "ABC Technologies",
  "document_date": "September 10, 2026",
  "item": "laptop computers",
  "quantity": "10",
  "amount": "850000"
}
```
```

Outline for this Session

- Agentic AI
- Local LLM Architecture
- Prompt Design
- **Structured Outputs**
- Failure Modes and Guardrails
- Case Study

Structured Outputs

- **Definition:** Structured output means an LLM returns information in a predefined, machine-readable format such as JSON, fixed fields, tables, or schema-based responses.
- It is useful when the output must be reliably stored, validated, routed, or passed to another system.
- Unlike free-form text, structured output is easier for software applications to interpret consistently.

Free-form output:

```
"This appears to be a
critical IT issue affecting
payroll."
```

Structured output:

```
{
  "category": "IT",
  "priority": "Critical",
  "affected_system":
"Payroll"
}
```

https://developers.openai.com/api/docs/guides/structured-outputs

Prompting an LLM to Return JSON

- **Example:** prompt:
  - Review the following purchase request.
  - Purchase amount: ₱1,200,000
  - Required threshold for human approval: ₱1,000,000
  - Determine whether the request requires human review.
  - Return the result as JSON with the following fields:
  - decision
  - reason
  - confidence

Expected Result:

```
{
  "decision": "HUMAN_REVIEW",
  "reason": "Purchase amount exceeds
₱1,000,000.",
  "confidence": 0.98
}
```

The LLM May Still Break the Format

- Potential Output:
  - `Here is my assessment:`
  - `Decision: Human Review`
  - `Reason: The amount exceeds the threshold.`
  - `Confidence: High`
- Desired Output:
  - `{`
  - `  "decision": "HUMAN_REVIEW",`
  - `  "confidence": "very high"`
  - `}`

Define the Output Schema

- **Definition:** A schema specifies what the LLM is allowed to return
- It defines required fields, data types, and allowed values.
- Optional and unknown values should be handled consistently.
- Clear schemas reduce ambiguity and make outputs easier to validate.

```
category
→ string
→ IT | Finance | HR | Procurement

priority
→ string
→ Low | Medium | High | Critical

affected_system
→ string or null

affected_users
→ integer or null
```

Prompting for Structured Output

- The prompt should define both the **task** and the required output **format**.
- Specify the fields to extract and the allowed categorical values.
- Do not allow the model to invent missing information.
- Require valid structured output, such as JSON, with minimal unnecessary style.

```
Analyze the request below.

Return valid JSON only.

Allowed category:
IT, Finance, HR, Procurement

Allowed priority:
Low, Medium, High, Critical

Use exactly these fields:
category
priority
affected_system
affected_users

If a value is not provided, return null.
Do not invent missing information.
```

Common Structured Output Failures

- Asking for JSON does not guarantee **valid data**
  - The model may return syntactically invalid JSON even when JSON is explicitly requested.
  - The model may use unexpected field names that do not match the application schema.
  - The model may return an invalid categorical value such as "Urgent" when only "Critical" is allowed.
  - The model may return the wrong data type, such as "two thousand" instead of the integer 2000.
  - The model may omit required fields or add unnecessary fields.
  - The model may infer unsupported information instead of returning null.
- Structured generation still **requires validation**.
- Examples:

Wrong field:

```
"department": "IT"
```

Invalid category:

```
"priority": "Urgent"
```

Wrong type:

```
"affected_users": "two thousand"
```

Structured Output vs Business Rules

- **Structured Output** and **Business Rules** are two different components
- Schema validation checks whether data is structurally valid.
- Business rules determine what action the application should take.
- The LLM interprets the input, while deterministic application logic should enforce critical rules.

LLM Output:

```
{
  "category": "IT",
  "priority": "Critical",
  "affected_system": "Payroll",
  "affected_users": 2000
}
```

Python Code:

```
IF priority = Critical
→ Require human review

IF affected_users > 1000
→ Escalate the case

IF affected_system is null
→ Request additional information
```

Validation

- Treat LLM output as a proposal, not guaranteed correct data.
- Validate fields, types, required values, and allowed categories before use.
- Invalid outputs should be rejected, retried, repaired, or sent for human review.
- Only validated output should enter databases or trigger actions.

What Is Pydantic?

- **Definition:** Pydantic is an open-source Python library used for data validation, parsing, and settings management using standard Python type hints.
- It checks fields, data types, allowed values, and optional values against a schema.
- Invalid data can be rejected before entering the application.
- Pydantic does not improve LLM accuracy; it ensures the output is structurally acceptable.
- It separates probabilistic LLM output from deterministic application logic.



https://github.com/pydantic/PYDANTIC

LLM-Generated JSON Can Still Be Wrong

- **Validation** prevents malformed or unsupported values from entering later workflow stages
- An LLM may return valid JSON such as:

```
{
  "category": "Technology",
  "priority": "Urgent",
  "affected_system": "Payroll",
  "affected_users": "two thousand"
}
```

- The JSON is syntactically valid, but the Pydantic may check:
  - Reject unexpected categories such as "Technology".
  - Reject unsupported priority values such as "Urgent".
  - Enforce numerical fields such as affected_users.

Structured Output

- **Exercise 4 — Structured Output**
- **Objectives**
  - Define the expected model output using a Pydantic schema.
  - Require the local LLM to produce JSON-compatible structured output.
  - Restrict important fields to approved values.
  - Validate the returned JSON before using it in the application.
- **Business Case**
  - The procurement application can no longer rely on free-form text such as:
  - “This seems to require human review.”
  - The application needs predictable fields that another Python function, database, or workflow can use.

Structured Output

- Expected Output

```
{
  "request_id": "PR-2026-001",
  "decision": "HUMAN_REVIEW",
  "reason": "The amount exceeds PHP 1,000,000.",
  "policy_applied": "AMOUNT_GT_1M",
  "next_action": "ROUTE_TO_HUMAN_APPROVER"
}
```

```python
#%% EXERCISE 4: STRUCTURED AND SCHEMA-BASED OUTPUT


import ollama

from pydantic import TypeAdapter, ValidationError

model_name = "gemma3:4b"


# ------------------------------------------------
# 1. DEFINE THE JSON SCHEMA
# ------------------------------------------------

purchase_review_schema = {
    "type": "object",

    "properties": {

        "request_id": {
            "type": "string"
        },

        "decision": {
            "type": "string",
            "enum": [
                "HUMAN_REVIEW",
                "STANDARD_REVIEW"
            ]
        },
```

Structured Output

```
==========================================
RAW STRUCTURED OUTPUT
==========================================
{"request_id": "PR-2026-001", "decision": "HUMAN_REVIEW", "reason":
"Purchase amount (PHP 1,200,000) exceeds PHP 1,000,000.",
"policy_applied": "AMOUNT_GT_1M", "next_action":
"ROUTE_TO_HUMAN_APPROVER"}

==========================================
PYDANTIC AND SCHEMA VALIDATION
==========================================
VALIDATION: PASSED

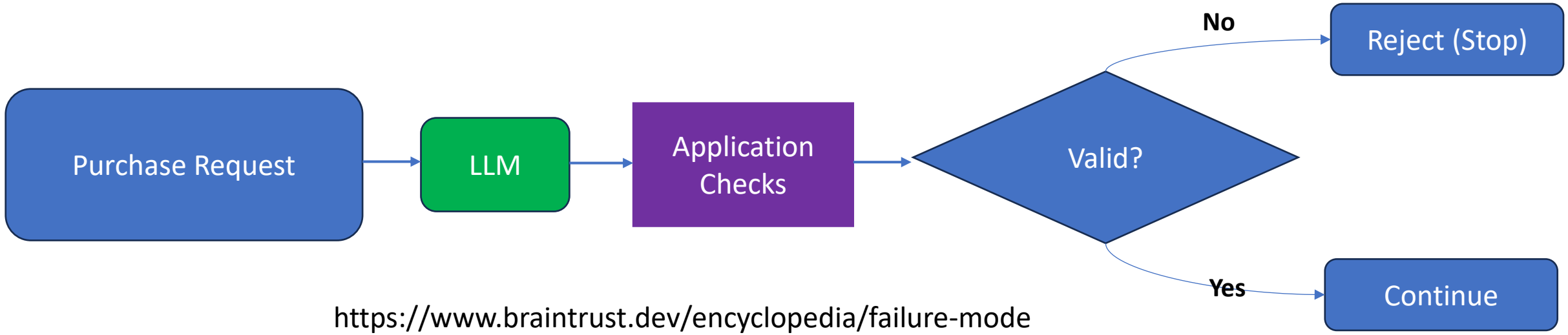
==========================================
VALIDATED PURCHASE REVIEW
==========================================
Request ID     : PR-2026-001
Decision       : HUMAN_REVIEW
Reason         : Purchase amount (PHP 1,200,000) exceeds PHP 1,000,000.
Policy Applied : AMOUNT_GT_1M
Next Action    : ROUTE_TO_HUMAN_APPROVER
```

Outline for this Session

- Agentic AI
- Local LLM Architecture
- Prompt Design
- Structured Outputs
- **Failure Modes and Guardrails**
- Case Study

Failure Modes

- **Definition:** An LLM **failure mode** is a specific, predictable pattern where an AI model produces an incorrect, unsafe, or undesired response due to its architecture, training data, or deployment setup
- LLMs may invent information, misunderstand rules, or produce invalid outputs.
- Required fields may be missing, incomplete, or incorrectly formatted.
- Agentic systems increase the risk because these outputs may trigger real actions.



https://www.braintrust.dev/encyclopedia/failure-mode

Guardrails

- **Definition:** LLM guardrails are programmatic policy enforcement and security controls that sit outside a large language model to inspect, validate, and restrict user prompts, retrieved data, and model outputs.
- They ensure AI systems operate within safe, ethical, and organizational boundaries rather than relying solely on the model's internal training

https://medium.com/data-science/safeguarding-llms-with-guardrails-4f5d9f57cff2

Failure Mode: Hallucination

- **Definition:** Hallucination occurs when an LLM generates information that is unsupported by the supplied input or evidence.
- **Purchase request supplied**
  - Request ID: PR-2026-001
  - Item: 20 laptop computers
  - Amount: PHP 1,200,000
  - Purpose: Replace obsolete laboratory computers.
- **Problematic LLM output**
  - DECISION: HUMAN_REVIEW
  - REASON: The amount exceeds the approval threshold.
  - SUPPLIER: ABC Technologies
  - QUOTATION_STATUS: Complete
- But neither the supplier nor quotation information was provided.

Failure Mode: Hallucination

- **Guardrail:** Add instructions such as:

- `Use only information supplied in the request.`
- `Do not invent missing information.`
- `If a value is not provided, return null.`

Failure Mode: Invalid Structured Output

- Even if we ask for JSON, the response may still be unusable.

Expected schema

```
decision:
HUMAN_REVIEW |
STANDARD_REVIEW

amount:
    number

reason:
    string
```

Invalid response

```
{
  "decision": "SEND TO MANAGER",
  "amount": "one point two
            million"
}
```

Handling Missing Information with Null Values

- The model should distinguish between information that is **present** and information that is **unknown**.
- Returning null is safer than inventing unsupported values.
- Explicit missing-information fields help the application decide whether clarification is required.
- This pattern supports later human-review and agentic workflow decisions.

```
Input:
“Our system is not working.”
A poor output would be:
{
  "affected_system": "Payroll",
  "affected_users": 500
}
A better output would be:
{
  "category": "IT",
  "priority": null,
  "affected_system": null,
  "affected_users": null,
  "missing_information": [
    "priority",
    "affected_system",
    "affected_users"
  ]
}
```

Other Common Failure Modes in LLMs

- **Format failure:** The model returns prose when JSON or another structured format is required.
- **Invalid values:** A categorical field may contain unsupported values.
- **Rule misunderstanding:** The model may apply the wrong business rule.
- **Overconfidence:** The model may provide a confident answer despite insufficient information.
- **Inconsistency:** Related fields may contradict each other.

Verification Before Action

- **Validate** the structure before using the output.
- Check whether required values are missing.
- Verify important decisions against deterministic business rules.
- Require evidence for high-impact decisions.
- Route uncertain or incomplete cases to human review.
- Log the model response and the final action taken.

Failure Modes and Guardrails

- **Exercise 5 — Safely Processing Purchase Requests**
- **Objectives**
  - Identify common LLM failure modes.
  - Use Pydantic to validate the basic structure of model output.
  - Apply deterministic checks for required fields and allowed values.
  - Detect missing source information before trusting the LLM.
  - Independently verify business rules in Python.
  - Route questionable outputs to human review rather than automatically accepting them.
- **Business Case**
  - The Purchase Request Review Assistant is now producing structured output. However, production AI applications must handle situations where the model, Produces invalid structured data, Omits required information, Returns unsupported values, Makes a decision inconsistent with a business rule, Invents information not contained in the request.
  - For this exercise, the system will process several purchase requests, including deliberately problematic cases.

Exercise 5 — Failure Modes and Guardrails

- Expected Output

Guardrail	Detects
Input validation	Missing amount or request ID
Ollama JSON schema	Guides structured generation
Pydantic	Invalid JSON/data structure
Allowed-value checks	Unsupported decisions/actions
Business-rule validation	Incorrect LLM decision
Human review	Uncertain or exceptional cases

```
Normal Request
Request ID: PR-2026-001
Amount: PHP 1,200,000

Standard Request:
Request ID: PR-2026-
002
Amount: PHP 250,000

Missing Amount:
Request ID: PR-2026-003
Amount: Not provided
```

Exercise 5 — Failure Modes and Guardrails

```python
#%% EXERCISE 5: FAILURE MODES AND GUARDRAILS

import re
import ollama

from pydantic import TypeAdapter, ValidationError

model_name = "gemma3:4b"


# ------------------------------------------------
# 1. DEFINE THE EXPECTED JSON SCHEMA
# ------------------------------------------------

purchase_review_schema = {
    "type": "object",

    "properties": {

        "request_id": {
            "type": "string"
        },

        "decision": {
            "type": "string",
            "enum": [
                "HUMAN_REVIEW",
                "STANDARD_REVIEW"
            ]
        },
```

```
========================================
PURCHASE REQUEST
========================================
Request ID: PR-2026-001
    Item: 20 laptop computers
    Amount: PHP 1,200,000
    Purpose: Replace obsolete laboratory computers.

LLM OUTPUT:
{
  "request_id": "PR-2026-001",
  "decision": "HUMAN_REVIEW",
  "reason": "The purchase amount exceeds PHP 1,000,000.",
  "policy_applied": "AMOUNT_GT_1M","next_action": "ROUTE_TO_HUMAN_APPROVER"
}

========================================
GUARDRAIL RESULTS
========================================
Input Complete       : YES
Pydantic Validation  : PASSED
Required Fields      : PASSED
Business Rule Check  : PASSED

========================================
FINAL WORKFLOW ACTION
========================================
Route purchase request to human approver.
```

Outline for this Session

- Agentic AI
- Local LLM Architecture
- Prompt Design
- Structured Outputs
- Failure Modes and Guardrails
- **Case Study**

Case Study 3 and Project Work

- **Objective:** Develop a controlled LLM application that interprets purchase requests, generates structured decisions, and applies Python guardrails before determining the next workflow step.
- A university wants to automate the initial screening of purchase requests. The AI assistant must classify requests according to the following business rules:
  - Amount above PHP 1,000,000 → HUMAN_REVIEW.
  - Amount at or below PHP 1,000,000 → STANDARD_REVIEW.
  - Missing supplier information → HOLD.

Request ID	Department	Item	Amount	Purpose	Supplier
PR-2026-001	Engineering Laboratory	20 laptop computers	1,200,000.00	Replace obsolete laboratory computers.	TechSource Systems
PR-2026-002	Engineering Laboratory	Laboratory printer	850,000.00	Replace damaged laboratory equipment.	None
PR-2026-003	Information Technology Office	Network switches	450,000.00	Replace aging network equipment.	Network Systems Corporation

Case Study 3 and Project Work

- Instructions:
  - Execute the provided Python code using Ollama and Qwen3.5:4b.
  - Compare free-form LLM responses with controlled workflow decisions.
  - Apply prompt engineering and generate structured JSON outputs.
  - Validate model outputs using Python business rules and guardrails.
  - Test the three purchase requests and examine the resulting workflow decisions.

Capstone Case 3: Build Your Own Controlled LLM Component

- **Objective:** Apply the concepts from the guided case study to implement an LLM-powered component for the group's proposed Agentic AI solution.
- Instructions:
  - Select an AI-supported activity from your Project Case 2 swimlane flowchart.
  - Define the input, expected output, and relevant business rules for the selected activity.
  - Develop a Python function that uses Ollama and Qwen3.5:4b to process the business input.
  - Design your own prompt and JSON schema for the selected business activity.
  - Implement Python validation and at least one deterministic business-rule guardrail.
  - Test the component using normal, incomplete, and invalid or adversarial inputs.

Outline for this Session

- Agentic AI
- Local LLM Architecture
- Prompt Design
- Structured Outputs
- Failure Modes and Guardrails
- Case Study